Exponential time hypothesis is false... but the verified algorithm is slower than trying all 2^n assignments at every practical size.

Alex Linhares
Argo Lab AGI; on leave from the Getulio Vargas Foundation
alex@argolab.org ORCID 0000-0002-4227-6879

November 5, 2026

Abstract

Placeholder. Item 09 writes the abstract (under 200 words; no number without a section that regenerates it). The title's sentence made precise: what is verified is a guarantee with existential constants and the exponent $n(\log\log n)^2/\sqrt{\log n}$, whose factor exceeds 1 for every $50 \le n < e^{5504}$, so the verified guarantee is weaker than the trivial $2^n \cdot \text{poly}$ guarantee at every practical size; the machine's running time is unknown because the constants are never computed; the manuscript's sharper exponents are unverified; the statement does refute ETH; the audit found no flaw.

1 Introduction

Placeholder. Item 09 drafts this section: the title's sentence made precise in the first paragraph (a guarantee, not a running time; existential constants never computed; $\rho(n) > n$ for every $50 \le n < e^{5504}$; the sharper exponents unverified); the one-paragraph summary; what is claimed and what is not; the stance (a careful reading by people who checked; no flaw found). Ends with what this does not claim".

2 The verified statement, read exactly

The whole of what is machine-checked about family 378 is one theorem, `exact_count_and_sat`, stated in the 220-line comparator file `lean/ComparatorChallenges/SubexponentialSAT.lean` of the collection (OpenAI, 2026) at commit `4eaee875`. The file carries the statement with a `sorry` for its proof (line 218); the solution tree proves the same statement, and the collection's comparator checks that the two agree (Section 4). The replay of Section 5 rebuilt that proof and found the three standard axioms and nothing else. This section reads the statement, definition by definition, and says what it asserts and what it does not. Line numbers are those of the comparator file; we quote nothing from memory.

2.1 The definitions, one by one

Formulas and the count (lines 13–51). A `Literal V` is a variable with a sign (lines 19–22); a `Clause V` is a `Finset` of literals and a `CNF V` a `Finset` of clauses (lines 24–25). An assignment is any function $V \to$ `Bool` (line 17). A literal evaluates to the Boolean "the assignment agrees with its sign" (lines 29–30), a clause to "some literal is true" (lines 32–33), a formula to "every clause is true" (lines 35–36). `WidthAtMost k F` says every clause has at most k literals as a set (line 38). Over a finite variable type, `satisfyingAssignments F` filters the universe of all assignments (lines 44–45), and `count F` is its cardinality (line 47): the number of satisfying assignments among all 2^n when $V =$ `Fin` n. The `Finset` choices change nothing in the meaning: a repeated clause or literal collapses, the empty clause is allowed and unsatisfiable, a tautological clause $\{x, \neg x\}$ is allowed and counts two toward the width. That is consistent with the manuscript's "Repeating a literal permits clauses with fewer than k literals" (p. 1, line 44).

The input encoding (lines 53–81). `encodeNat n` is n ones followed by a zero (line 55): unary. `frame` doubles a bit string, prefixing each bit with a one, and ends it with a zero (lines 57–58). `encodeList enc` prefixes each element's frame with a one and ends the list with a zero (lines 60–62). A literal becomes the number $2 \cdot \text{var} + [\text{positive}]$, at most $2n - 1$ (lines 64–65); a clause is its sorted list of literal numbers, a formula its sorted list of clauses (lines 67–71). `encodeCNF F` is `encodeNat n` followed by the formula's list of lists of unary numbers (lines 73–77), and `formulaBitLength F` is the length of that bit string (line 79). Two things follow. The number of variables n is part of the input, so a variable that occurs in no clause is still counted. And the length is polynomial: a clause of $j \le k$ literals costs $2j(4n+1)+4$ bits, so

$$L \;=\; \texttt{formulaBitLength}\, F \;\le\; n + 2 + m\left(2k(4n+1)+4\right) \;=\; O(k\,n\,m)$$

for m clauses. The solution itself proves $(L+1)^d \le (8k+7)^d\,(n+|F|+1)^{2d}$ (`SqrtLogSpace/Statement.lean`, lines 9–14). Emulating the definitions (`reports/scratch/eth_audit/check_encoding.py`), a formula with $n = 125$, $m = 500$, $k = 4$ encodes to $L =$ 1,006,223 bits and one with $n =$ 1,000, $m =$ 4,000, $k = 3$ to 47.9 million, both under the bound. Nothing in the encoding is exponential; the unary numbers are linear in n.

The machine (lines 83–128). `PrimitiveMachine` (lines 92–99) is a finite object: a number `tapeCount` of tapes and a number `stateCount` of states, an input tape and a start state, a table `scan` saying which tape each state reads, and a table `transition` from `Fin stateCount` $\times$ `Option Bool`—a domain of $3 \cdot$ `stateCount` points—into either a halting answer (a `Bool`) or a next state with one action. The action type is Mathlib's `Turing.TM0.Stmt (Option Bool)`: move the head left or right, or write one of the three symbols `none`, `some false`, `some true`, five values in all (`Mathlib/Computability/TuringMachine/PostTuringMachine.lean` lines 118–120 and `Tape.lean` lines 421–423 at Mathlib `d13f23b7`, the clone's pinned revision). A configuration `Cfg` (lines 103–105) is a state, halted or not, and one Mathlib `Tape` per tape index (a head symbol and two blank-padded halves, `Tape.lean` lines 408–414, blank = `none`). `step` (lines 107–119) returns `none` in a halted state; otherwise it reads the head of the one tape the state scans, looks the pair up in the table, and either halts with the answer, or writes one symbol or moves that one head by one cell and changes state. `init` (lines 121–124) puts the input on the input tape with the head at its first bit and every other tape blank (`Tape.mk`$_1$, `Tape.lean` line

494). A Lean function on a finite domain is, whatever term defines it, a finite table; so a step is one lookup in a table of $3 \cdot$ `stateCount` rows, and no field of the structure can carry unbounded computation.

Traces (lines 87–90). `Trace step t a c` is an inductive proposition with two constructors: `refl`, a trace of length 0 from a to a itself, and `cons`, which extends a trace from b by one successful step $a \mapsto b$. So it holds exactly when c is reached from a by t successful applications of `step`, and a trace of length 0 to a different configuration is impossible. Since `step` is a function, the run is deterministic.

Space monitoring (lines 147–189). `LocatedCfg` (lines 149–153) pairs a configuration with, per tape, the head's current position and the least and greatest positions seen so far, as integers; `locatedInit` starts every head at 0 with the input tape's range covering the input (lines 155–159); `headDisplacement` is -1, $+1$ or 0 for a move left, a move right or a write (lines 161–164); `monitor` (lines 166–182) applies the same table lookup to update the position and the running minimum and maximum of the scanned tape, and nothing else; `locatedStep` is `step` on the configuration followed by `monitor` (lines 184–186). `visitedSpace` (lines 188–189) is the sum over tapes of upper $-$ lower $+ 1$: the number of cells whose head position was ever visited. `countWord a` is the binary expansion of a, least significant bit first, with `[false]` for 0 (line 147), and `toNat` (lines 7–9) decodes such a word.

The exponent and the bound (lines 130–141).

```
runtimeScale n = n * (Real.log (Real.log (n + 16)))^2 /
Real.sqrt (Real.log (n + 16))
```

(lines 132–134, transcribed; the casts to $\mathbb{R}$ omitted), that is

$$\rho(n) \;=\; \frac{n\,\bigl(\ln\ln(n+16)\bigr)^2}{\sqrt{\ln(n+16)}},$$

with `Real.log` the natural logarithm. The manuscript fixes the same base: "Throughout, log denotes the natural logarithm" (p. 1, lines 48–49; again at p. 7, lines 326–327). The shift by 16 keeps $\ln\ln$ defined and positive at every $n \geq 0$ ($\ln 16 = 2.77$), so the theorem can be stated for all n with no proviso. Then

```
runtimeBound C A d L n = C * (L + 1)^d * 2^(A * runtimeScale n)
```

(lines 136–139, transcribed), a real number, with the real-exponent power. We write $\rho(n)$ for `runtimeScale n` throughout the paper.

2.2 The theorem, and the order of its quantifiers

Lines 194–217 state, for every k with $2 \le k$:

$\exists$ `machine`, `countTape`, `answerTape`, $C, A, S \in \mathbb{R}$, $d, e \in \mathbb{N}$,
$0 < C \ \wedge\ 0 < A \ \wedge\ 0 < S \ \wedge\ 0 < d \ \wedge\ 0 < e \ \wedge$
$\forall\, n \in \mathbb{N},\ \forall\, F : \texttt{CNF}\ (\texttt{Fin}\ n),\ \texttt{WidthAtMost}\ k\ F \ \rightarrow$
$\exists$ `time` $\in \mathbb{N}$, `final` : `LocatedCfg machine`,
`Trace` (`locatedStep machine`) `time` (`locatedInit machine` (`encodeCNF` F)) `final`
(lines 201–202)
$\wedge$ the count tape holds `countWord (count F)` (203–204)
$\wedge$ the answer tape holds the bit $[0 <$ `count` $F]$ (205–206)
$\wedge$ the halting state carries that bit (207)
$\wedge$ `toNat` (`countWord` (`count` F)) $=$ `count` F (208)
$\wedge$ $|$`countWord` (`count` F)$| \le n + 1$ (209)
$\wedge$ `time` $\le C\,(L+1)^d\, 2^{A\,\rho(n)}$, $L =$ `formulaBitLength` F (210–211)
$\wedge\ \forall$ prefix of the trace, `visitedSpace` $\le S\,(L+1)^e$ (212–217).

Three features of the order matter.

The machine and the five constants come after k and before n and F. They may depend on k and on nothing else; inside the universal quantifier only `time` and `final` are chosen. So for each k one fixed finite machine serves every formula of every size, and C, A, S, d, e are the same for all of them: the standard $O_k(1)$ reading. The table's size is $O_k(1)$ in the same sense, with no explicit function of k.

There is no lower bound on n. The manuscript's Theorem 1.1 and Proposition 6.4 are stated for $n \ge 20$ (p. 2, lines 56–57; p. 43, lines 2474–2475), and the solution's auxiliary `exact_count_eth` keeps $20 \le n$ (`SqrtLogSpace/Final.lean`, lines 104–109); the exported theorem does not. The $+16$ in $\rho(n)$ is what makes that possible.

The constants are existential. The statement asserts that values of C, A, S, d, e exist; it names none. The proof obtains them from existential lemmas and never evaluates them: at the point where the large-input run is assembled, the exponent constant is set to $A = 256 + A_t$ with $A_t \ge 1$ taken from a leaf contract (`SqrtLogSpace/NativeLargeEntry.lean`, line 51; `SqrtLogSpace/NativeLeafContract.lean`, lines 792–793), and the degree and the polynomial constants are built the same way. So the proof as written has $A \ge 257$, and no closed value of any of the five. The witness machine is explicit in one respect: it is compiled from a program over 80 one-bit registers and 1,220 tapes, with `countTape` $= 256$ and `answerTape` $= 1216$ (`SqrtLogSpace/Final.lean`, lines 98–101); its state count is a finite type's cardinality that nothing computes. Section 4 is the inspection of the proof; this section only reads what the statement fixes and what it leaves open.

2.3 The exponent, in numbers

Everything the title says rests on one function. With $y = \ln(n + 16)$,

$$\frac{\rho(n)}{n} \;=\; \frac{(\ln y)^2}{\sqrt{y}},$$

which rises from 0.64 at $n = 1$, crosses 1 between $n = 49$ and $n = 50$, peaks at $16/e^2 = 2.165$ where $\ln y = 4$, i.e. $y = e^4$, $n = e^{e^4} - 16 \approx 10^{23.7}$, and returns below 1 only where $(\ln y)^2 = \sqrt{y}$ again, at $y = 5503.7$, i.e. $n \approx e^{5504} \approx 10^{2390}$. Table 2.1 gives the values and Figures 2.1 and 2.2 draw them; every number is printed by `paper3/scripts/runtime\_scale.py` (run under `python3 -I`; it imports nothing from the clone and computes in y, so that no n beyond floating-point range is ever formed). The crossing at 50 is a fact about integers: $\rho(49)/49 = 0.9994$ and $\rho(50)/50 = 1.0027$. In the limit $\rho(n)/n \to 0$, since $(\ln y)^2/\sqrt{y} \to 0$: the exponent is $o(n)$, and that is a theorem in our companion Lean (Appendix A, A.2). Both facts are true at once, and the second is the one the title is about: the function is $o(n)$, and it is above n on every size anyone will ever write down.

Table 2.1: The verified exponent relative to n. $\rho(n) = n(\ln\ln(n+16))^2/\sqrt{\ln(n+16)}$, the comparator's `runtimeScale` (lines 132–134), natural logarithms. The verified bound is $C(L+1)^d\, 2^{A\rho(n)}$; at $A = 1$ its exponential factor is $2^{(\rho(n)/n)\cdot n}$, against 2^n for trying every assignment. Regenerate: `python3 -I paper3/scripts/runtime_scale.py`.

n	$\rho(n)/n$	$\rho(n)$
1	0.6443	0.64
20	0.8606	17.21
49	0.9994	48.97
50	1.0027	50.14
100	1.1146	111.46
125	1.1495	143.69
10^3	1.4229	1422.88
10^6	1.8550	1.85×10^6
10^{12}	2.0956	2.10×10^{12}
$10^{23.7}$	2.1654	—
10^{100}	1.9497	—
10^{1000}	1.2490	—
$e^{5504} \approx 10^{2390}$	1.0000	—
10^{3000}	0.9403	—

2.4 The title's claim, made precise

The title says the verified algorithm is slower than trying all 2^n assignments at every practical size. Read against the statement, that is the following, and nothing more.

1. **What is verified is a guarantee, not a running time.** The theorem says the machine halts within $C(L+1)^d\, 2^{A\rho(n)}$ steps. It does not say how many steps the machine takes on any input, and no implementation of the machine exists to be timed. A bound above 2^n does not say the machine is slow; it says the proof promises nothing better than 2^n there.

2. **The constants C, A, S, d, e are existential and never computed.** The comparator asserts their existence; the proof builds them from existential lemmas (above, with $A \geq 257$ as constructed) and states no value. The machine's state count is likewise never computed.

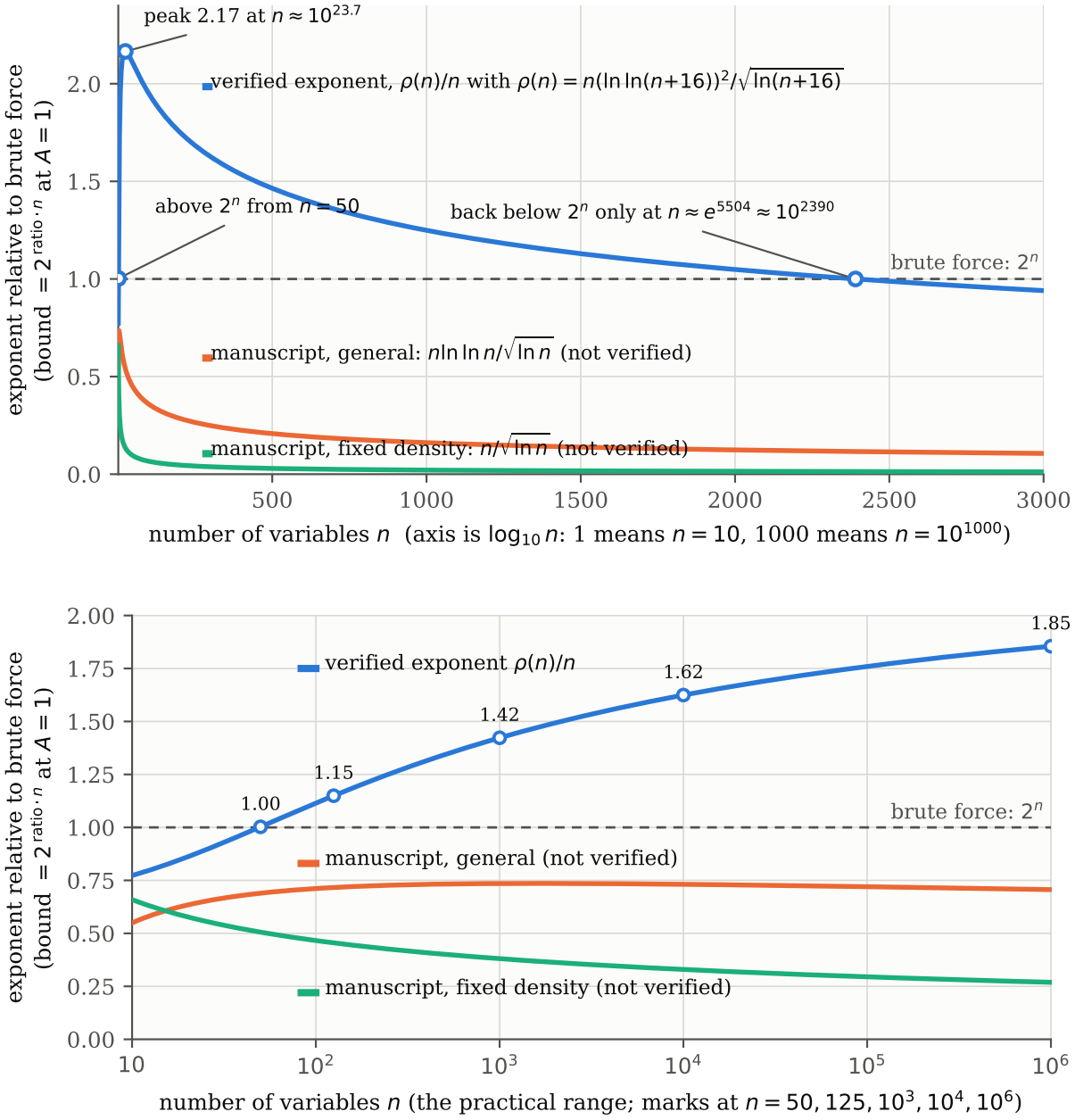


Figure 2.1: The verified exponent against brute force. Top: $\rho(n)/n$ over the whole range, the axis in $\log_{10} n$ from $n = 10$ to $n = 10^{3000}$; the dashed level is 2^n; beneath it the manuscript's two sharper exponents, $n \ln\ln n/\sqrt{\ln n}$ (Theorem 1.1, general) and $n/\sqrt{\ln n}$ (fixed clause density), which are not in the Lean library (Section 3). Bottom: the same three curves over the practical range $n = 10$ to 10^6, with $\rho(n)/n$ marked at $n = 50, 125, 10^3, 10^4, 10^6$. All curves at the constant $A = 1$; the verified A is unknown. Regenerate: `python3 -I paper3/scripts/rho_over_n.py`.

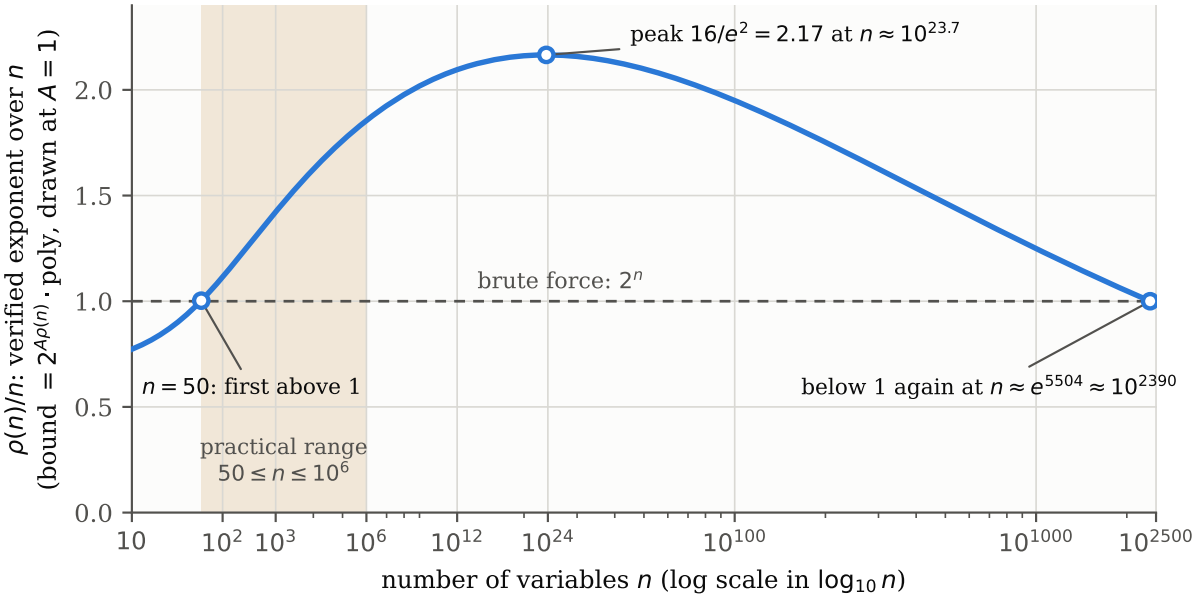


Figure 2.2: $\rho(n)/n$ alone, on a logarithmic axis in $\log_{10} n$ from $n = 10$ to $n = 10^{2500}$, with the level 1 (brute force, 2^n), the first integer above the level ($n = 50$), the peak ($16/e^2 = 2.17$ at $\ln(n+16) = e^4$, $n \approx 10^{23.7}$), the return below the level ($n \approx e^{5504} \approx 10^{2390}$), and the practical range $50 \le n \le 10^6$ shaded. Regenerate: `python3 -I paper3/scripts/runtime_scale.py`.

3. **With $A = 1$, the verified exponent exceeds n for every $50 \le n < e^{5504}$.** That is Table 2.1: the exponential factor of the bound is at least 2^n from 50 variables up to about 10^{2390}, and $2^{2.17\,n}$ at its worst. With the true A, whatever it is, the factor is $2^{A\rho(n)} \ge 2^{\rho(n)}$ once $A \ge 1$, and the proof's own construction has $A \ge 257$; so nothing is known that would put the bound below 2^n at any practical size.

4. **Hence, at every practical size, the verified guarantee is weaker than the guarantee of trying all assignments.** Trying all 2^n assignments and checking each against the m clauses counts the models in $2^n \cdot \mathrm{poly}(n+m)$ steps on any reasonable machine, with constants one can write down. The verified guarantee is $C(L+1)^d\, 2^{A\rho(n)}$ with $\rho(n) > n$ on $50 \le n < e^{5504}$ and C, A, d unknown. For every $C \ge 1$, $A \ge 1$ and $d \ge 1$ the verified bound is above 2^n on every n in that range, and the only values on record are the proof's, which has $1 \le C$ and $1 \le A$ in the large-input run's own statement (`SqrtLogSpace/NativeLargeEntry.lean`, line 12) and $A \ge 257$ in its construction. A proof with $A < 1$ would be a different proof; none is on record.

None of this is a defect of the formalisation or of the proof; it is what an $O_k(1)$ statement with a $2^{o(n)}$ exponent of this shape means when the exponent's constants are not tracked. The manuscript itself calls $\rho(n)$ "the weaker exponent" (p. 2, line 63), and its sharper exponents are not in the Lean library (Section 3). One more detail of base: the solution's internal ρ (`SqrtLogSpace/Asymptotics.lean`, lines 10–11) is in base 2 and is related to `runtimeScale` by `runtimeScale` $n \le 4\,\rho_2(n)$ for $n \ge 20$ (lines 190–191); the base changes the exponent by a constant factor that A absorbs, so nothing above depends on it.

2.5 What the statement does establish

For each fixed $k \geq 2$, a deterministic machine that, on the canonical encoding of any k-CNF on n variables with m clauses, halts with the exact number of satisfying assignments in binary on a named tape, within $C(L+1)^d\, 2^{A\rho(n)}$ steps, where $L = O(k\,n\,m)$ is polynomial in $n+m$ and $\rho(n) = o(n)$; and, because a `Finset` of clauses of width at most k has at most $\sum_{j \leq k} \binom{2n}{j} = O(n^k)$ members (2,604,376 at $n = 125$, $k = 3$), the whole bound is $2^{o(n)}$ in n alone. That is a deterministic $2^{o(n)} \cdot \mathrm{poly}(n+m)$ algorithm for #k-SAT in the standard sense, and therefore for k-SAT; it refutes the Exponential Time Hypothesis (Impagliazzo and Paturi, 2001) as stated. The statement reads as the manuscript's verified form, the weaker exponent of its Proposition 6.4 (p. 43, lines 2474–2476), without the $n \geq 20$ proviso. We found no field, definition or quantifier through which the statement could be kernel-true and mean something weaker than this; the audit it summarises is `reports/scratch/eth_audit/statement\_audit.md`. Our own restatement of the statement as a Lean `Prop`, character for character over a verbatim copy of the machine model, is (Appendix A, A.1); it is never proved and never an axiom there (Section 8).

2.6 What this does not claim

- **No running time of any machine.** Nothing here measures or estimates how long the verified machine, or any program, takes on any input. The title compares two *guarantees*; Section 7 is the only place in the paper where anything is timed, and it times ordinary solvers, not this machine.
- **No statement about the proof.** That the statement means what we say it means is a reading of 220 lines; whether the proof of it is sound is Section 4 and Section 5, which found no flaw. The one proof-side fact quoted here, $A = 256 + A_t$, is a line of the solution read for the value it assigns, not a claim about the argument.
- **No value of the constants.** $A \geq 257$ is what the proof writes down, not what the machine needs; a different proof of the same statement could have any $A > 0$. The statement itself fixes nothing.
- **Nothing about the manuscript's sharper exponents**, which are not in the Lean library and are Section 3's subject.
- **No claim about sizes beyond e^{5504}.** Past $n \approx 10^{2390}$ the verified factor is below 2^n at $A = 1$; we say nothing about that range except that it is not a practical one.

3 What the manuscript claims beyond the verified statement

The manuscript (OpenAI, 2026) is 67 pages; the comparator statement is 220 lines. Section 2 read the statement. This section reads the manuscript's claims one by one and says, for each, whether it is what the Lean theorem asserts, whether it follows from the Lean theorem by an argument that exists only on paper, or whether it rests on a bound that the Lean library does not contain. The collection's own scope note draws the line in two sentences, which we quote in full (`lean/docs/378.md`, lines 9 and 11):

> "For fixed $k \geq 2$, Lean proves that a deterministic machine exactly counts satisfying assignments of a canonically encoded k-CNF on n variables in time $(L+1)^{O_k(1)}\, 2^{O_k(n(\log\log n)^2/\sqrt{\log n})}$, where L is the encoding's bit length. At every step, the machine has visited at most $(L+1)^{O_k(1)}$ tape cells in total."
>
> "For m clauses, the October 25 manuscript gives the sharper time bound $(n+m)^{O_k(1)}\, 2^{O_k(n\log\log n/\sqrt{\log n})}$. When $m \leq Dn$ for fixed $D > 0$, this improves to $(n+m)^{O_{k,D}(1)}\, 2^{O_{k,D}(n/\sqrt{\log n})}$."

Everything below is an elaboration of that note against the manuscript's text and the library's files. Table 3.2 is the summary.

3.1 Three statuses, and what "paper-only" means

We tag every claim with one of three words.

- **Verified.** The claim is the comparator statement `exact_count_and_sat`, or one of its conjuncts, or a theorem of the solution tree that the replay of Section 5 compiled and whose axioms it printed. Nothing else is called verified.
- **Supported.** The claim follows from the verified statement by an argument that is on paper only: a reduction of a few lines, or a citation. The argument is not in the Lean library, and we have not checked it; what we have checked is that the only property of the counter it uses is one the verified statement provides, and that the exponent it inherits is therefore $\rho(n)$, not a sharper one.
- **Paper-only.** The claim rests on a bound that the Lean library at commit `4eaee875` does not contain: Theorem 1.1's two sharper exponents, and whatever is derived from them.

The brief for this section had two tags, verified and paper-only; the middle one is ours, because the manuscript's corollaries are of two different kinds and a single word would have put Corollary 1.3 (a counting-preserving encoding into the verified counter) beside Corollary 1.4 (a bound whose proof calls the fixed-density exponent by name). In the brief's partition, "supported" is a sub-case of "paper-only": not in the library. In no case does "paper-only" mean "wrong". We did not read the manuscript's proofs for correctness; the tag records where a claim is established, not whether.

3.2 The sharper exponents of Theorem 1.1

Theorem 1.1 (p. 2, lines 56–62) states two running-time bounds for the same deterministic algorithm: $(n+m)^{O_k(1)}\, 2^{O_k(n\log\log n/\sqrt{\log n})}$ for every k-CNF on $n \geq 20$ declared variables, and $(n+m)^{O_{k,D}(1)}\, 2^{O_{k,D}(n/\sqrt{\log n})}$ when $m \leq Dn$ for a fixed $D > 0$; both with $(n+m)^{O_k(1)}$ bits of space (line 60). The sentence that follows the theorem is the one the title of this paper is about (p. 2, lines 63–67):

> "The rational interpolation argument in Section 6.4 gives the weaker exponent $O_k(\rho(n))$, where $\rho(n) = n(\log\log n)^2/\sqrt{\log n} = n/(\log n)^{1/2+o(1)}$. Appendix D proves both bounds in Theorem 1.1 by tensor contraction."

Proposition 6.4 (p. 43, lines 2474–2476) is the weaker bound as a statement of its own: for each fixed $k \geq 2$, the interpolation algorithm counts in $(n+m)^{O_k(1)}\, 2^{O_k(\rho(n))}$ bit operations and $(n+m)^{O_k(1)}$ bits of space, for $n \geq 20$.

Which of the three is in the Lean. The comparator's `runtimeScale` (lines 132–134 of the comparator file) is $n(\ln\ln(n+16))^2/\sqrt{\ln(n+16)}$, the squared form. The solution's own base-2 exponent `rho` (`SqrtLogSpace/Asymptotics.lean`, lines 10–11) is $n(\log_2\log_2 n)^2/\sqrt{\log_2 n}$, Proposition 6.4's ρ with the base made explicit, and `runtimeScale_le_four_rho` (lines 190–191) bounds the comparator's exponent by four times it for $n \geq 20$. The solution's sparsification parameters are Proposition 6.4's to the letter: the proof selects $\lambda_n = \lceil \log_2 n \rceil$, $\nu_n = \lceil \log_2 \lambda_n \rceil$ and $s = \lfloor \sqrt{\lambda_n}/\nu_n^2 \rfloor$, "the largest nonnegative integer satisfying $s^2\nu_n^4 \leq \lambda_n$" (p. 43, lines 2480–2486), and `Asymptotics.lean` lines 14–20 define `sparsifyLambda`, `sparsifyNu` and `sparsifySchedule` as exactly those three quantities in natural-number arithmetic. So the verified exponent is the weaker one, and the verified proof is organised around Section 6.4's parameters; what Appendix D proves is not in the library in any form we could find (Section 3.6). The general exponent $n\log\log n/\sqrt{\log n}$ and the fixed-density exponent $n/\sqrt{\log n}$ are **paper-only**: proved at p. 57 (lines 3222–3246) and p. 54 (lines 3045–3064) respectively, by the contraction theorem of Appendix D.

Both bounds from one algorithm. The manuscript notes that a single algorithm attains both paper-only bounds, switching on $m \leq n\nu$ (p. 57, lines 3247–3250). That algorithm is not the verified one either.

The three exponents, in numbers. Table 3.1 puts the three exponents side by side, each divided by n and each at constant 1. The verified ratio is the one of Table 2.1. The general paper-only ratio, $\ln\ln n/\sqrt{\ln n}$, never reaches 1: it peaks at $2/e = 0.736$ where $\ln n = e^2$, that is $n = e^{e^2} \approx 1{,}618$, and falls on both sides. The fixed-density ratio $1/\sqrt{\ln n}$ decreases from 0.578 at $n = 20$. So, at constant 1, both paper-only guarantees sit below 2^n at every size, and the verified one sits above 2^n from $n = 50$ to $n \approx 10^{2390}$. That is the whole content of the title's qualifier "verified": the manuscript's exponents would make a different sentence, and they are not what the kernel checked. Two cautions apply to the table equally. The constants in front of every exponent are $O_k(1)$ or $O_{k,D}(1)$ and are not computed anywhere, in the manuscript or in the Lean; and the manuscript's exponents are undefined below $n = e$ and are stated for $n \geq 20$, while the verified one is stated for all n (the $+16$). Figure 2.1 draws the same three curves.

The abstract's form. The abstract states the running time as $(n+m)^{O_k(1)}\, 2^{n/(\log n)^{1/2+o(1)}}$ (p. 1, lines 6–9). Since $(\ln\ln n)^2 = (\ln n)^{o(1)}$, the verified exponent is of that form too, and the manuscript says so at lines 64–66. We tag the abstract's form **verified**, with the remark that its $o(1)$ absorbs exactly the factor the three rows of Table 3.1 differ by: at $n = 10^6$ it is $(\ln\ln n)^2 = 6.9$ for the verified exponent, $\ln\ln n = 2.6$ for the general one and 1 for the fixed-density one.

3.3 The corollaries

Finding and sampling a satisfying assignment (p. 2, lines 68–71). "When $\#F > 0$, the standard self-reduction finds a satisfying assignment by fixing one variable at a time and checking that the

Table 3.1: The three exponents of the manuscript divided by n, at constant 1: the verified $\rho(n)/n = (\ln\ln(n+16))^2/\sqrt{\ln(n+16)}$ (comparator lines 132–134); Theorem 1.1's general exponent $\ln\ln n/\sqrt{\ln n}$ (p. 2, line 59); its fixed-density exponent $1/\sqrt{\ln n}$ (p. 2, line 62). Natural logarithms throughout (p. 1, lines 48–49). The row e^{e^2} is the peak of the general ratio $(2/e)$. Regenerate: `python3 -I paper3/scripts/beyond_claims.py`.

n	verified $\rho(n)/n$	general $\ln\ln n/\sqrt{\ln n}$	density $1/\sqrt{\ln n}$
20	0.861	0.634	0.578
50	1.003	0.690	0.506
125	1.150	0.717	0.455
10^3	1.423	0.735	0.380
$e^{e^2} \approx 1{,}618$	1.472	0.736	0.368
10^6	1.855	0.706	0.269
$10^{23.7}$	2.165	0.541	0.135

remaining count is positive. Choosing each bit according to the two conditional counts instead gives an exactly uniform random satisfying assignment," with polynomial expected overhead and a citation. **Supported**: the self-reduction calls the verified counter at most n times on formulas of the same width, and the verified statement provides the exact count it needs. The reduction is two sentences of prose and is not in the library.

Corollary 1.2, Max-k-SAT within εm (p. 2, lines 77–94). A randomised algorithm finds an assignment satisfying at least $\mathrm{OPT}(F) - \varepsilon m$ clauses with probability $1 - 2^{-\Omega(n)}$ in time

$$(n+m)^{O_{k,\varepsilon}(1)}\, 2^{O_{k,\varepsilon}(\rho(n))}$$

(line 80). The corollary is stated with $\rho(n)$ itself, the verified exponent. Its proof samples $O(n/\varepsilon^2)$ clauses, builds a threshold circuit of size $O_{k,\varepsilon}(n)$, reduces it to a 3-CNF with $O_{k,\varepsilon}(n)$ variables by a cited lemma, and calls Theorem 1.1 with binary search and self-reduction (lines 82–94). **Supported**: the exponent matches the verified one because $\rho(cn) = O_c(\rho(n))$, and every call is to a fixed-width counter on $O(n)$ variables; the sampling, the circuit and the reduction are on paper. Its randomness is in the reduction, not in the counter.

Corollary 1.3, independent sets, colourings, perfect matchings (p. 2–3, lines 100–120). The number of independent sets and, for fixed q, of proper q-colourings of a graph on v vertices and e edges in $2^{o(v)}$ time, and the number of perfect matchings in $2^{o(v+e)}$ time, all in polynomial space. The encodings are explicit: one variable per vertex and a 2-clause per edge (lines 107–112); q variables per vertex with exactly-one and no-equal-colour clauses, a q-CNF on qv variables (lines 112–115); one variable per edge and a linear-size circuit for "exactly one selected edge meets each vertex", reduced to 3-CNF with $O(v+e)$ variables, whose gate variables are determined by the edge bits so that the count is preserved (lines 116–119). **Supported**: the conclusion is $2^{o(\cdot)}$, which needs only $\rho(n) = o(n)$ (Appendix A, A.2), and each encoding is counting-preserving into a fixed-width CNF on $O(v)$ or $O(v+e)$ variables. None of the three encodings is in the library.

Corollary 1.4, a circuit lower bound for E^{NP} (p. 3, lines 121–127). "The fixed-density case of Theorem 1.1 gives a circuit lower bound for E^{NP}" (line 121): a language $L \in \mathrm{E}^{\mathrm{NP}}$ and a constant

$c > 0$ with $C_L(n) > c\,n\sqrt{\log n}$ for infinitely many n, where $C_L(n)$ counts non-input fan-in-two gates (lines 123–127). The proof is Appendix C.1 (p. 51–52, lines 2925–2967), Williams's witness-circuit argument with a local reduction, and it invokes the paper-only exponent in so many words: "Put $a(t) = \sqrt{\log_2 t}$. Testing whether the count in the constant-density part of Theorem 1.1 is positive decides 3-SAT with V variables and at most $5V$ clauses in time $V^b\,2^{AV/a(V)}$ for fixed $A, b > 0$" (p. 52, lines 2949–2950); the threshold $s_n = \lfloor \eta n \lfloor a(n) \rfloor \rfloor$ is chosen against that a (line 2953), and the constant c comes out as $\sqrt{\eta/(d \log 2)}$ (line 2967). **Paper-only**: the quantitative bound $c\,n\sqrt{\log n}$ is tied to $2^{AV/\sqrt{\log V}}$, which the library does not contain. What the same argument would give from the verified exponent is a question about the manuscript's proof, which we do not examine (Section 3.8).

Corollary C.1, $\mathrm{E}^{\mathrm{NP}} \not\subseteq \mathrm{SIZE}(O(n))$ **(p. 51, lines 2912–2916).** "Williams proved that if ETH is false, then E^{NP} does not have linear-size circuits [Wil13, Theorem 6.1]. Apply this implication to the deterministic subexponential-time 3-SAT algorithm supplied by Theorem 1.1." **Supported**: the only input is a deterministic $2^{o(n)}$ 3-SAT decision, which the verified statement provides on its answer tape for $k = 3$; the implication is a citation. The manuscript's own scope sentence applies: the corollary "does not assert a superpolynomial circuit lower bound for NP" (lines 2923–2924).

Corollary C.2, inverting constant-locality maps (p. 52, lines 2974–2977). For a Boolean map whose every output bit reads at most d input bits, the preimage count $|G^{-1}(\vec{b})|$ and a preimage in deterministic time $(n+m)^{O_d(1)}\,2^{O_d(\rho(n))}$ and polynomial space, for $n \geq 20$. Stated with $\rho(n)$. The proof writes one clause per forbidden local assignment, a CNF of width at most d on the original n variables with at most $2^d m$ clauses whose satisfying assignments are exactly the preimages, and applies Theorem 1.1 with $k = \max\{2, d\}$ (lines 2979–2988). **Supported**: the encoding is on paper; the counter and the exponent are the verified ones. The remark that follows, on Goldreich's candidate one-way functions (p. 53, lines 2994–2999), inherits the same status and the manuscript's own limit, "This does not rule out one-wayness against polynomial-time algorithms" (line 2999).

3.4 Appendix D: the contraction route

Appendix D, "A sharper bound for exact counting" (p. 53–64, from line 3001), proves Theorem 1.1 "by contracting Boolean tensor networks. The two running times reflect the number of wires in the networks" (lines 3002–3003). Theorem D.1 (p. 53, lines 3017–3028) is the contraction theorem: a bipartite tensor network with at most Cs edges, maximum degree at most $\lceil s^{9/10} \rceil$ and entries in $(\mathbb{Z}/2^{P_0}\mathbb{Z})[V]/(V^\ell)$ is contracted in $(s+L)^{O(1)}\,2^{O(s/\sqrt{\log s})}$ bit operations and polynomial space. The fixed-density proof (p. 54, lines 3045–3064) encodes a formula as a network with $O_k(n+m)$ edges and applies Theorem D.1 with $s = n + m$; the arbitrary-density proof (p. 57, lines 3222–3246) combines the sparsification of Section 6 with Lemma D.2 (p. 54, line 3069) and reaches the exponent $O_k(n \log\log n/\sqrt{\log n})$ at line 3243. Section D.4 changes the logarithm convention for its own proof, "log denotes a base-two logarithm and ln denotes a natural logarithm" (p. 57, lines 3251–3252). All of this is **paper-only**: no declaration about tensor networks, contraction, Hermite recovery or packets exists in the family's 3,977 files, by the identifier search of Section 3.6. We make no statement about Appendix D's argument; we did not read it for correctness.

3.5 The hypotheses of §1.1.1 and the fine-grained remarks of §1.1.3

§1.1.1 (p. 4, lines 187–191). "Theorem 1.1 refutes ETH, SETH, Gap-ETH, and NSETH. Indeed, testing whether the exact count is positive or zero decides both satisfiability and unsatisfiability in deterministic $2^{o(n)}(n+m)^{O_k(1)}$ time for every fixed k; deciding satisfiability also solves the Gap-ETH promise problem. The deterministic algorithm also refutes the bounded-error randomized versions of ETH [...] and SETH [...], and of Gap-ETH." And §1.1.2 (p. 5, lines 211–212): "Our algorithm (Theorem 1.1) solves #3-SAT deterministically in $2^{o(n)}\,\mathrm{poly}(n,m)$ time and therefore refutes #ETH." **Supported**, and by the shortest argument in the manuscript: the verified statement is a deterministic machine whose answer tape carries $[0 < \texttt{count}\ F]$ and whose count tape carries the count, within $2^{o(n)}$ times a polynomial, for each fixed $k \geq 2$; each hypothesis is the negation of that. The hypotheses themselves are not defined in the library (the strings "ETH" and "SETH" occur in the collection's Lean only in two files of a different family, `lean/OAI/Combinatorics/ParityLifts/`, which take an ETH-like proposition as a hypothesis and are not in the closure). Section 6 states each hypothesis with its quantifiers and says exactly how the verified statement negates it; the two sentences the manuscript adds at lines 191–194, on unconditional lower bounds for restricted proof systems and on Williams's 2016 MA-SETH protocol, are context, not claims about the result.

§1.1.3 (p. 5, lines 213–243). After a list of lower bounds proved from ETH or SETH (lines 213–225), the manuscript writes: "Our refutation of ETH and SETH removes one basis for the hardness claims above. Some of the same lower bounds follow from weaker hypotheses that may still hold" (lines 226–227), names the Orthogonal Vectors Conjecture (lines 229–232) and the hypothesis that unrestricted CNF-SAT admits no $2^{(1-\delta)n}\,\mathrm{poly}(n,m)$ algorithm, adding: "Theorem 1.1 gives a running-time guarantee for each fixed clause width k, while this hypothesis allows the width to grow with n" (lines 235–236). These are **paper-only** remarks in the sense that nothing about any other hypothesis or any reduction is in the library; they are also the manuscript's own statement of what the result does not do, and Section 6 rests on them. The one positive transfer, Set Splitting with sets of size at most k by one variable per element and two clauses per set (lines 237–243), is **supported** in the same sense as Corollary 1.3: an encoding on paper into the verified counter, inheriting $\rho(n)$. The caveat at p. 2, lines 95–99, that $2^{o(n)}$ bounds "do not transfer under general polynomial-time reductions," is not a claim but a limit the manuscript places on its own result; it is quoted again in Section 6.

3.6 What the library carries beyond the comparator statement

"Paper-only" is defined against the whole library at commit `4eaee875`, not only against the comparator file, so we say what else the library proves about this family. `SqrtLogSpace/Final.lean` (201 lines) states five theorems, each followed by a `#print axioms` directive (lines 197–201), and the replay's build log carries the kernel's answer for all five: the three standard axioms (`paper3/replay/logs/run2/axioms\_in\_build.txt`).

- `compiled_exact_count_and_sat` (line 10) and `exact_count_and_sat` (line 74): the comparator statement, first over the explicit 80-register, 1,220-tape program and then over the machine it compiles to (Section 2).

- `exact_count_eth` (lines 104–147): the same conclusion with the polynomial factor written as $(n + |F| + 1)^d$ instead of $(L + 1)^d$, the exponent as $A \cdot \texttt{rho}\ n$ in base 2, and the proviso $20 \leq n$ (line 109); its proof takes $2d$, $2e$ and $4A$ from the comparator theorem's constants (line 132). This is Proposition 6.4's $(n + m)^{O_k(1)}\, 2^{O_k(\rho(n))}$ form in the library, so the manuscript's polynomial factor is verified as well as its exponent; it is **verified**, and it is still the weaker exponent.

- `sat_polynomial_space` (lines 149–183), for $k \geq 3$: the answer bit satisfies `answer = true` $\leftrightarrow$ `SAT.Satisfiable F`, the decision form made explicit.

- `original_sat_main` (line 184): the proposition `SqrtLog.Statement.MainTheorem` (`SqrtLog/StatementCanonical.lean`, line 37), a decision-only target for every width $k \geq 3$ with the same `runtimeBound`, stated there as a `Prop` "no inhabitant is supplied here" and inhabited in `Final.lean`.

Beyond these, an identifier search over the family's 3,977 `.lean` files, by the read-only script `paper3/scripts/beyond\_claims.py`, finds no occurrence of `SimpleGraph`, `PerfectMatching`, `Coloring`, `IndependentSet`, `CircuitSize`, `OrthogonalVectors`, `MaxSat`, `TensorNetwork`, `HermiteInterp`, `ENP`, `SIZE`, `SetSplitting`, `GapETH` or `NSETH` as an identifier: none of the corollaries, none of Appendix D, and none of the hypotheses is formalised. The comparator's JSON, `lean/ComparatorChallenges/SubexponentialSAT.json`, names one theorem, `exact_count_and_sat`, and no definition. A grep is necessary and not sufficient (Section 4); here it is used for absence, where a false negative would require a corollary to be formalised under none of the words its own statement uses.

3.7 The table

3.8 What this does not claim

- **No judgement on the paper-only proofs.** Appendix D, Appendix C.1 and the two sharper bounds of Theorem 1.1 were located and quoted, not read for correctness. "Paper-only" means not in the Lean library at commit `4eaee875`; it does not mean wrong, and nothing here suggests it.

- **No judgement on the supported arguments either.** The reductions behind Corollaries 1.2, 1.3, C.1, C.2 and the Set Splitting transfer are standard in kind and short, and we checked only which property of the counter they use. Whether each is correct as written is the manuscript's, not ours.

- **Nothing about what the verified exponent would yield through Corollary 1.4's argument.** A reader can rerun Appendix C.1 with $a(t)$ replaced by the verified rate; we did not, because it is a question about a paper-only proof.

- **No constants.** Every $O_k(1)$ and $O_{k,D}(1)$ in this section is as unknown as the comparator's C, A, S, d, e; the comparisons of Table 3.1 are at constant 1 and say nothing about any machine's running time.

- **Nothing beyond the collection.** Whether the sharper bounds have since been formalised elsewhere, or the manuscript revised, is not examined; what is read is the commit and the dated manuscript.

Table 3.2: Every claim of the manuscript examined in this section: where it is stated (page of `paper.pdf`, lines of the committed extract), its status in the sense of Section 3.1, and the one-line reason. Generated, with every page and fragment checked against the extract and the clone, by `python3 -I paper3/scripts/beyond_claims.py`.

claim	stated	status	why
Abstract: time $(n+m)^{O_k(1)}\,2^{n/(\log n)^{1/2+o(1)}}$, polynomial space	p. 1, 6–11	verified (as a form)	$\rho(n) = n/(\log n)^{1/2+o(1)}$ too (p. 2, lines 64–66); the $o(1)$ absorbs the difference between all three exponents
Theorem 1.1, general: $(n+m)^{O_k(1)}\,2^{O_k(n \log\log n/\sqrt{\log n})}$	p. 2, 56–59	paper-only	proved in Appendix D.3 (p. 57, lines 3222–3246); the comparator has $(\log\log)^2$; `docs/378.md` line 11
Theorem 1.1, $m \le Dn$: $(n+m)^{O_{k,D}(1)}\,2^{O_{k,D}(n/\sqrt{\log n})}$	p. 2, 60–62	paper-only	proved in Appendix D.2 (p. 54, lines 3045–3064); no density case in the Lean; `docs/378.md` line 11
Theorem 1.1: $(n+m)^{O_k(1)}$ bits of space	p. 2, 60	verified	the conjunct `visitedSpace` $\le S(L+1)^e$ at every prefix, $L = O(knm)$; as $(n+\lvert F\rvert+1)^e$ in `Final.lean` 104–147
The weaker exponent $O_k(\rho(n))$, Proposition 6.4	p. 2, 63–66	verified	the comparator statement (Section 2), with the $n \ge 20$ proviso removed; Prop. 6.4 at p. 43, lines 2474–2476
Proposition 6.4's parameters $\lambda_n = \lceil \log_2 n \rceil$, $\nu_n = \lceil \log_2 \lambda_n \rceil$, $s = \lfloor \sqrt{\lambda_n}/\nu_n^2 \rfloor$	p. 43, 2480–2486	verified (the same parameters)	`Asymptotics.lean` lines 14–20 define `sparsifyLambda`, `sparsifyNu`, `sparsifySchedule` identically
"Appendix D proves both bounds in Theorem 1.1 by tensor contraction"	p. 2, 67	paper-only	no tensor-network declaration in the library (Section 3.6)
Self-reduction finds a satisfying assignment; exactly uniform sampling	p. 2, 68–71	supported	calls the verified counter n times; the reduction is prose; cites [JVV86]
Corollary 1.2: Max-k-SAT within εm, randomised, $2^{O_{k,\varepsilon}(\rho(n))}$	p. 2, 77–81	supported	stated with $\rho(n)$ itself; its proof (lines 82–94) samples clauses and reduces a threshold circuit to 3-CNF, on paper
Caveat: $2^{o(n)}$ bounds do not transfer under polynomial-time reductions	p. 2, 95–99	a caveat, not a claim	quoted in Section 6
Corollary 1.3: independent sets, q-colourings in $2^{o(v)}$; perfect matchings in $2^{o(v+e)}$	p. 2, 100–103	supported	counting-preserving encodings (lines 107–120), one variable per vertex, q per vertex, one per edge plus a circuit; needs only $\rho(n) = o(n)$
Corollary 1.4: $C_L(n) > c\,n\sqrt{\log n}$ for some $L \in \mathrm{E}^{\mathrm{NP}}$	p. 3, 123–127	paper-only	its proof (p. 51–52, lines 2925–2967) uses "the constant-density part of Theorem 1.1" (line 2949), the $2^{AV/\sqrt{\log_2 V}}$ bound
§1.1.1: Theorem 1.1 refutes ETH, SETH, Gap-ETH, NSETH and their bounded-error randomised forms	p. 4, 187–191	supported	the one-line reason is the manuscript's own: a positive-or-zero test on the verified count, in $2^{o(n)}(n+m)^{O_k(1)}$; Section 6
§1.1.2: Theorem 1.1 refutes #ETH	p. 5, 211–212	supported	the verified machine is a deterministic $2^{o(n)}\,\mathrm{poly}(n,m)$ #3-SAT counter
§1.1.3: "removes one basis for the hardness claims above"; OVC and unrestricted CNF-SAT may still hold	p. 5, 226–236	paper-only (remarks)	statements about other hypotheses and reductions; nothing of them is in the library; Section 6
§1.1.3: Set Splitting with sets of size $\le k$ in subexponential time	p. 5, 237–243	supported	one variable per element, two clauses per set (lines 239–243), on paper; inherits $\rho(n)$
Corollary C.1: $\mathrm{E}^{\mathrm{NP}} \not\subseteq \mathrm{SIZE}(O(n))$	p. 51, 2912–2916	supported	needs only a deterministic $2^{o(n)}$ 3-SAT decision (line 2915) plus Williams 2013, Theorem 6.1, cited
Corollary C.2: inversion and exact preimage counting of constant-locality maps, $2^{O_d(\rho(n))}$	p. 52, 2974–2977	supported	stated with $\rho(n)$ and $n \ge 20$; a width-d CNF on the same n variables, $k = \max\{2, d\}$ (lines 2984–2988)
Appendix D: Theorem D.1 (tensor contraction), Lemma D.2, both proofs of Theorem 1.1	p. 53, 3001–3028	paper-only	proofs at p. 54 (3045–3064) and p. 57 (3222–3250); D.4 uses base-2 logarithms (p. 57, lines 3251–3252)

4 The proof, inspected

Section 2 read the theorem; this section reads the proof, as far as reading can go. The question is not whether the mathematics is right—the kernel decides that, and Section 5 reports what the kernel said when we ran it—but whether anything in the source could make the kernel's acceptance mean less than it appears to mean: an axiom, a `sorry`, a trusted evaluation, an unsound instance, a patched Mathlib, a statement that differs from the challenge's, or a cost model of the proof's own that leaks into the theorem. We found none of these. What we did find is that the repository offers no recorded evidence that the closure type-checks at all—no `#print axioms` output, no continuous integration, no second kernel for this family—which is why Section 5 exists. The last subsection joins the two.

4.1 Method

The clone of `openai/math` at commit `4eaee875` is treated as untrusted data: every file was read with `cat`, `sed`, `grep` and `python3 -I`, and nothing in it was built, imported or executed for this section. Two scripts from the audit of 2026-11-05 do the mechanical part: `closure.py` follows `import` lines from the solution module, strips comments (nested block comments included) and counts keyword occurrences in what remains; `defdiff.py` normalises whitespace and compares each declaration of the challenge file with the library's declaration of the same name. Both were re-run for this section, and every number below is from that fresh run; `paper3/scripts/proof_inspection.py` re-runs them, performs the remaining checks, and regenerates Table 4.1. The inspection is of the spine of the proof—the final file, the compiler, the entry lemmas, the constants—and of the whole closure by keyword. The arithmetic beneath the spine was not read (Section 4.9).

4.2 The closure

The comparator's JSON names one solution module, `OAI.Algorithms.SubexponentialSAT.Main`, a sixteen-line aggregator that imports `SqrtLog.ExecutionNativeFinal` and `SqrtLogSpace.Final` and declares nothing (`Main.lean`). Its import closure inside the repository is **3,091 modules**, every one with the prefix `OAI.Algorithms.SubexponentialSAT`, totalling **550,551** raw lines; the largest file is 2,873 lines (`SqrtLogSpace/DisjointReuse/SparsePriorityProgram.lean`). The closure stops at **50** Mathlib modules and one module of Lean's own library, `Lean.Elab.Tactic.Omega`, imported by `SqrtLogSpace/FiniteFallback.lean` (line 14) and `SqrtLogSpace/DisjointReuse/SparseCharging.lean` (line 5); it is a tactic module, and the closure defines no tactic, macro or syntax of its own (Section 4.3). Nothing is imported from any other family of the collection or from any of the repository's other dependencies.

The family's directory holds **3,977** `.lean` files, so **886** are outside the closure. They are not reachable from the collection's root module either: `lean/OAI.lean` imports the family only at its line 49, `import OAI.Algorithms.SubexponentialSAT.Main`, and the lakefile's default target is that root (`lean/lakefile.lean`, line 89, `@[default_target] lean_lib OAI`). They are therefore never built by the default target and bear on nothing: dead files, not a hole. For completeness, a raw search of the 886 for `sorry`, `axiom`, `native_decide` and `admit` finds nothing, with or without comments.

4.3 The keyword sweep

Table 4.1 gives the counts over the 3,091 files with comments stripped. The ones that matter are the zeros. There is no `axiom`, no `sorry`, no `admit`; no `native_decide` and no `ofReduceBool`, so nothing is proved by trusting the compiled evaluator; no `implemented_by`, `extern`, `unsafe` or `partial`, so no definition has a hidden implementation and every recursion is one the kernel checks (nine `termination_by` and five `decreasing_by` clauses); no `elab`, `macro`, `syntax` or `simproc`, so there is no metaprogramming that could rewrite goals outside the kernel's view. The only `set_option`s are two `set_option quotPrecheck false` (`SqrtLog/LeafProgramPreparedRecoveryQuery.lean`, line 15, and `SqrtLog/LeafProgramPreparedRecoveryWitness.lean`, line 14), which switch off a pre-check on local notation and have no bearing on soundness; there is no `maxHeartbeats`, `maxRecDepth` or `autoImplicit` option anywhere in the closure, and the lakefile sets `autoImplicit` to `false` for the whole package (`lean/lakefile.lean`, line 9).

Nine declarations are `opaque`, and every one carries a body:

- three recovery functions: `queryRecovery` at line 59 of `SqrtLog/LeafProgramPreparedRecoveryQuery.lean`, with its `:= by` at line 86; `rawRecovery` and `fixedRecovery` at lines 178 and 198 of `SqrtLog/LeafProgramPreparedRecoveryWitness.lean`;
- six `private opaque` constants of a subtype `{ports // ports = ...}`, each with an explicit value: line 22 of `SqrtLog/LeafProgramQueryCleanup.lean` and five others.

An `opaque` with a body is a definition whose unfolding is hidden from elaboration; the kernel still checks the body against the type, so nothing is assumed. The 1,048 `noncomputable` declarations, the eight `open Classical`, and the four explicit uses each of `Classical.choice` and `propext` are the classical reasoning the comparator permits. The 148 `instance`s and 40 `local instance`s are `Fintype`, `Decidable` and algebraic instances; an instance cannot be "inconsistent" without an axiom, and there is none. The 14,034 lines with `decide` are kernel-evaluated decision procedures, not the native evaluator.

4.4 The statement is the challenge's

The solution file does not import the challenge file: `SqrtLogSpace/Final.lean` begins (lines 1–3) with `NativeLargeEntry`, `SqrtLogSpace.Statement` and `SqrtLog.StatementCanonical`. The library re-declares every definition the statement uses under the same fully qualified names, spread over seven files: the SAT vocabulary in `SAT.lean`, the bit arithmetic in `BinaryArithmetic.lean`, the nine encoding functions in `Encoding.lean`, the machine in `Computation.lean`, `runtimeScale` and `runtimeBound` in `SqrtLog/StatementScale.lean`, the space instrumentation in `SqrtLogSpace/VisitedTape.lean` and `countWord` in `SqrtLogSpace/EncodingBounds.lean` (line 17, the challenge's line 147). The definition diff compares **35** declarations: **33** are identical character for character after whitespace normalisation, and the remaining two, `runtimeScale` and `runtimeBound`, differ only in that the challenge writes `noncomputable def` where the library opens a `noncomputable section` (`SqrtLog/StatementScale.lean`, line 15)—a difference the kernel does not see. The theorem statement itself, from `theorem exact_count_and_sat`

Table 4.1: The inspection in numbers. Comments stripped where a keyword is counted. Regenerate: `python3 -I paper3/scripts/proof_inspection.py`.

quantity	value	script
modules in the closure of `Main.lean`	3,091	`closure.py`
raw lines in those modules	550,551	`closure.py`
external imports: Mathlib modules / core modules	50 / 1	`closure.py`
`.lean` files under the family / outside the closure	3,977 / 886	this script
`axiom`, `sorry`, `admit`, `native_decide`, `ofReduceBool`	0, 0, 0, 0, 0	`closure.py`
`implemented_by`, `extern`, `unsafe`, `partial`	0, 0, 0, 0	`closure.py`
`elab`, `macro`, `syntax`, `simproc`	0, 0, 0, 0	`closure.py`
`set_option` (both `quotPrecheck false`)	2	`closure.py`
`opaque` (all with bodies)	9 (9)	`closure.py`, this script
`noncomputable`	1,048	`closure.py`
explicit `Classical.choice` / `propext` / `open Classical`	4 / 4 / 8	`closure.py`
`instance` / `local instance`	148 / 40	`closure.py`
`termination_by` / `decreasing_by`	9 / 5	`closure.py`
statement definitions compared / identical / identical up to `noncomputable`	35 / 33 / 2	`defdiff.py`, this script
theorem statement, characters (normalised / raw), identical	1,124 / 1,353	this script
`#print axioms` commands: closure files / lines; family files / lines	8 / 19; 17 / 35	this script
recorded `depends on axioms` outputs in the clone	0	this script
challenge JSONs: total / `enable_nanoda` false / true / absent	418 / 413 / 2 / 3	this script
CI workflow files (`lock-prs.yml` only)	1	this script
manifest packages / patch files / patches for Mathlib	42 / 23 / 0	this script

to `:= by`, is **identical** in the two files, 1,353 characters raw and 1,124 after normalisation, at line 194 of the challenge and line 74 of `SqrtLogSpace/Final.lean`; the challenge's proof is the single word `sorry` at its line 218.

This textual match is a sanity check and not the guarantee. The comparator (Lean FRO, 2026) compares the two statements at the level of exported constants, with every declaration the statement mentions matched, so a library definition that differed from the challenge's under the same name would make the comparator fail rather than pass silently. What the text confirms is that there is no such difference to find, and that the shape Section 2 read is the shape the library proves.

4.5 The spine of the proof

The machine. The theorem's machine is `GuardedCompiler.primitive program` with the count tape `256` and the answer tape `1216` (both `: Fin 1220`; `SqrtLogSpace/Final.lean`, line 100), where `program` is `Native.countingProgram cutoff k (NativeLeafProgram.command k)` (line 40), a `BitProgram.Program 80 1220 labels` (line 11): a first-order program over 80 one-bit registers, 1,220 Boolean stacks and a fi-

nite label set, with twelve instruction kinds (`BitProgram.lean`, lines 10–26). `primitive` (`SqrtLogSpace/GuardedCompiler.lean`, line 131) is a `noncomputable abbrev`: it takes the transition table `transition p` (lines 30–47, one case per instruction) and the stack selector `selected p` (lines 14–17), feeds them to the stack compiler's controller, and renames the finite state type to `Fin stateCount` through `Fintype.equivFin` (line 132). The machine is thus a concrete table defined by pattern matching on the program, composed with a classical renaming of states; the theorem is existential, so the noncomputability costs it nothing, and it is the reason no running time can be read off the statement (Section 2).

The time bound, and the constants. For `cutoff < n`, `NativeLargeEntry.run` (`SqrtLogSpace/NativeLargeEntry.lean`, lines 11–33) supplies a trace of length at most `runtimeBound C A d L n` (line 27) with $1 \le C \wedge 1 \le A$ (line 12). Its exponent constant is `256 + At` (line 51), where `At` comes from the leaf program's uniform contract (`NativeLeafContract.uniform_contract`, line 39; the contract gives $1 \le$ `At`, `SqrtLogSpace/NativeLeafContract.lean`, line 793), and its cutoff is `max 20 (max Nleaf Ns)` (line 50), with `Ns` obtained non-constructively from an eventual inequality (`Filter.eventually_atTop`, line 48). The $2^{A \cdot \mathtt{runtimeScale}\, n}$ factor is assembled by `runtime_of_tree_and_leaf` (line 46) from the DFS visit bound of `Native.outer_from_setup` (`SqrtLogSpace/NativeOuter.lean`, lines 53–56: visits at most a square polynomial in n times $2^{n/\mathtt{sparsifySchedule}\, n}$) and the leaf time. For $n \le$ `cutoff`, `Native.entry_small_primitive` (`SqrtLogSpace/NativeEntry.lean`, lines 81–100) gives `time` $\le 28700\,(L+1)^2$ (line 92) through the finite scanner of `SqrtLogSpace/FiniteFallback.lean`. The two branches are folded by `runtime_with_fallback` and `space_with_fallback` (`SqrtLogSpace/NativeOuterBudgets.lean`, lines 297 and 317) into one triple, and the witnesses the final theorem exhibits are `C + 28700`, `A + 1`, `S + 5022741`, `d + 2`, `e + 2` (`SqrtLogSpace/Final.lean`, line 41). So the proof as written has an exponent constant of at least 258 in front of $\rho(n)$, a polynomial degree `d` that depends on k (line 51: `db` $+ (4(k-2)+2) +$ `df` $+ 2$) and a leading constant of at least 28,701; none of these is in the statement, which asserts only that they are positive, and none of `At`, `db`, `df` is computed anywhere. This confirms, from the proof side, what Section 2 said from the statement side: the constants are unknown, and the *smallest* they could be is already far from 1.

Where a hidden cost model would live, and why it cannot. A proof of a running-time bound for a machine it compiles from a higher-level program could, in principle, prove the bound against a cost function of its own and leave the kernel to check only that function's arithmetic. That is not what happens here. The compile lemmas conclude in the comparator's own vocabulary: `GuardedCompiler.primitive_execution` (`SqrtLogSpace/GuardedCompiler.lean`, lines 158–162) turns a source run of `n` steps into `RunsIn (primitive p).step (7 * n)`, with the overhead `7` proved as a theorem (`compiler_overhead`, lines 152–153), and `CompileSpace.compile_word_answer_space` (`SqrtLogSpace/CompileSpace.lean`, lines 125–143) produces a `Computation.Trace (locatedStep (primitive p)) time ...` with `time` $\le 7 \cdot$ `sourceSteps` and `visitedSpace intermediate` $\le s($`bound` $+$ `input.length` $+ 15)$ on every prefix (lines 134, 135 and 143); `primitive_located_step` (`SqrtLogSpace/GuardedCompiler.lean`, line 600) is the per-step form. `Trace`, `locatedStep` and `visitedSpace` are the challenge's definitions (its lines 87–90 and 149–

189), and `time` in the final theorem is literally the length of a `Computation.Trace` of `PrimitiveMachine.step` (its line 201). No cost function of the compiler's appears in the statement; a wrong cost model would make these lemmas unprovable, not the theorem false. This is the structural point on which the rest of the paper leans: whatever the proof's internal accounting, the number the theorem bounds is the number of steps of the comparator's machine.

Correctness of the count. `Native.outer_from_setup` (`SqrtLogSpace/NativeOuter.lean`, lines 50–51) yields `final.stacks ABI.countStack = countWord (SAT.count F)` and `final.stacks ABI.answerStack = [decide (0 < SAT.count F)]` from the canonical DFS execution, and `Native.counting_primitive_from_entry` (line 73) transports both to the primitive machine's tapes; `countWord_toNat` and `countWord_count_length` supply the decoding and length clauses of the statement. These are theorems about the concrete step function. The arithmetic that makes them true—sparsification, the ring and moment computations, the DFS—is the content of the other three thousand files, and the only claim made about it here is the one the kernel makes: it elaborates (Section 5).

4.6 Toolchain, Mathlib, and the build hooks

The toolchain is fixed (`lean/lean-toolchain`: `leanprover/lean4:v4.34.1`; `lean/lakefile.lean` line 8: `fixedToolchain := true`). The manifest lists 42 packages; Mathlib is pinned to upstream revision `d13f23b723b8a846827a245b89c10fc7d3f11612` at `leanprover-community/mathlib4`, and the audit of 2026-11-05 confirmed that revision on GitHub as the upstream commit that bumps the toolchain to `v4.34.1`. The repository carries 23 compatibility patches under `lean/patches/`, **none for Mathlib**; every one is for a third-party package (`AINTLIB`, `PrimeNumberTheoremAnd`, `carleson`, `iut`, ...) that this closure does not import. The lakefile has two hooks that run arbitrary IO when Lake configures or updates the package: a `run_cmd` block (lines 290–304) that evaluates dependency declarations with `unsafe env.evalConstCheck` and calls `preparePatchedDependencies`, which clones and patches eleven dependencies, and a `post_update` hook (line 306 onward) that verifies or applies the patches for the rest. They touch only `.lake/packages`, and only non-Mathlib packages; they are one reason the inspection did not build, and the reason the replay of Section 5 runs inside a sandbox with the network on.

4.7 What the repository does not record

Three things one would expect beside a verified refutation of a forty-year-old hypothesis are absent, and their absence is a fact about evidence rather than about the proof.

- **No axiom audit.** `SqrtLogSpace/Final.lean` ends with five `#print axioms` commands (lines 197–201), and seven other files of the closure carry fourteen more (nineteen commands in eight files; the family's directory holds seventeen such files, but nine of them are among the 886 never built). Their output exists only while the build runs and is recorded nowhere: a search of the whole clone for the string `depends on axioms` finds nothing.

- **No continuous integration.** `.github/workflows/` holds one file, `lock-prs.yml`, which locks pull-request conversations. Nothing builds or checks the Lean.
- **No second kernel for this family.** The comparator's JSON (`lean/ComparatorChallenges/SubexponentialSAT.json`) names the one theorem, an empty `definition_names`, the three permitted axioms `propext`, `Quot.sound`, `Classical.choice`, and `enable_nanoda: false`; the registry `lean/formalization.yaml` (lines 1683–1685) points at the same declaration and file. Of the collection's 418 challenge JSONs, 413 set `enable_nanoda` to `false`, two to `true`, and three omit the key. With it off, the collection's own check of this family rests on one kernel implementation. The comparator's README, as fetched by the audit, describes the check as: build challenge and solution in a sandbox, export both with `lean4export`, match each named theorem's statement with every declaration it uses, confine the proof to the permitted axioms, and have the Lean kernel accept it; with nanoda on, the exported environment is replayed through an independent kernel in Rust as well (nanoda contributors, 2026). Section 9 records the versions.

Two smaller consistency facts, for the record. The manuscript's source is dated "17 October 2026, revised 25 October 2026" (`preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/build/main_full_version.tex`, line 25); the collection's history file mentions the family once, in the sentence that names the working group at whose request the materials were released (`history.md`, line 18), and its only "Formalization Fix" entry (line 7) concerns another family. And the scope note `lean/docs/378.md` states the verified exponent as $n(\log\log n)^2/\sqrt{\log n}$ (line 9), which is what the library proves and what Section 3 compares with the manuscript's.

4.8 The trust boundary, joined with the replay

The audit's verdict listed what the proof's trust rested on before anything was run: (i) OpenAI's unpublished build, since nothing showed the closure type-checks; (ii) the absence of any recorded axiom output; (iii) the Lean kernel of `v4.34.1` and Mathlib `d13f23b7` as upstream; (iv) the comparator run, not reproduced; (v) no second kernel for this family. Table 4.2 says which of the five Section 5 moved.

After the replay, then, the proof's trust rests on: the Lean kernel of `v4.34.1` *and* nanoda agreeing on the exported dependencies of the theorem; `lean4export` exporting what the environment holds; upstream Mathlib at the pinned revision; and the textual, not export-level, match between the solution's statement and the challenge's. Everything the inspection could find by reading, the build and the two kernels confirmed; nothing the inspection could not settle by reading has been settled by anything else.

4.9 What this does not claim

- **The arithmetic was not read.** The inspection followed the spine from `SqrtLogSpace/Final.lean` down to the compile lemmas and swept the whole closure by keyword; the sparsification, ring and DFS content of the other three thousand files was not read. The argument above is that its correctness is kernel-enforced, not that we verified it.

Table 4.2: The trust boundary before and after the replay of Section 5.

item	status after the replay
(i) the build	**moved**: the 3,091-module closure built from source in our sandbox at the pinned toolchain, exit 0, no `error`, no `sorry`.
(ii) the axiom output	**moved**: `#print axioms` of the comparator's theorem printed `[propext, Classical.choice, Quot.sound]`, and the nineteen in-build axiom lines say the same; the output is on record.
(iii) kernel and Mathlib	**moved in part**: Mathlib was fetched from upstream at the pinned revision and not patched, so "as upstream" is now observed rather than assumed; the Lean kernel of `v4.34.1` is still trusted as upstream, now beside a second kernel (v).
(iv) the comparator run	**not moved**: the statement-level match through `lean4export` was not performed; the match this paper relies on is the textual one of Section 4.4.
(v) a second kernel	**moved**: `nanoda_lib` checked the exported dependencies of the theorem, 95,176 declarations, with no errors and the three axioms; what this adds and what it does not is stated in Section 5.

- **Grep is necessary, not sufficient.** Zero `sorry` and zero `axiom` by text search cannot show that a file elaborates; only the build does (Section 5). Conversely a keyword count is over source text at one commit, and a declaration can be unsound only through the mechanisms searched for; we searched for the ones we know.
- **The textual statement match is not the comparator's.** The comparator matches exported constants; we matched source text after whitespace normalisation and found it identical. A difference the normalisation hides would have to be invisible to the kernel as well, but that is an argument, not a run.
- **Nothing about the constants' values.** The bounds $A \geq 258$ and $C \geq 28{,}701$ are what the proof's witnesses are built from; they are lower bounds on what this proof exhibits, not statements about any machine's running time, and a different proof of the same statement could exhibit different ones.
- **Nothing about the Lean kernel, Mathlib or the toolchain.** All three are trusted as upstream at the pinned versions; the second kernel of Section 5 is evidence about the first, not a proof of either.
- **Nothing about what the theorem says.** That is Section 2; a sound proof of a weak bound is a sound proof of a weak bound.

5 Independent replay

The repository holds no recorded `#print axioms` output for the family, no continuous integration that builds its Lean, and `enable_nanoda: false` in the comparator's JSON (Section 4). We therefore built the solution's import closure ourselves, in isolation, at the toolchain the repository pins, and recorded what the kernel reports. This section says exactly what was run, where,

and what came out; the scripts are quoted verbatim in `paper3/replay/README.md`, and the logs are under `paper3/replay/logs/`.

5.1 What was run, and where

A fresh copy of the clone (commit `4eaee875`) was built inside a `bwrap` (bubblewrap) sandbox on the project's machine: the system directories bound read-only, a private `tmpfs` home with nothing of the real home or of this repository visible, separate PID, IPC and UTS namespaces, the whole sandbox confined to eight cores (`taskset -c 0-7`) at the lowest scheduling priority, and the network on, because the repository's `lakefile` clones its 41 dependencies at build time and the Mathlib object files come from the Mathlib cache. Inside, `elan` was installed into the private home and fetched the pinned toolchain from the repository's own `lean-toolchain` file; then, in the clone's `lean/` directory, `lake exe cache get`, `lake build OAI.Algorithms.SubexponentialSAT.Main` (the solution's closure, not the collection's default target, which builds all 721 families), and `#print axioms` of the one theorem the comparator names, `OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.exact_count_and_sat`, through `lake env lean`. Only text left the sandbox; nothing built inside it was executed outside it.

Two runs were needed. The first, started at 01:03 on 2026-11-05, failed at once at the build step because its command line carried `-j 8`, an option this version of Lake does not have (the whole of its build log is the one line `error: unknown short option '-j'`); nothing of the family was compiled. The second, started at 01:16 with that option removed and the core cap moved to `taskset`, is the run reported below. Both scripts, both versions, and the launch command are in the README; the first run's logs are kept beside the second's.

5.2 What came out

The build finished with exit code 0 in 33 minutes of wall clock on eight cores (01:16:25 to 01:49:31), compiling **3,091** modules with the prefix `OAI.Algorithms.SubexponentialSAT`—exactly the import closure the inspection of Section 4 enumerated, the same 3,091 names with an empty set difference both ways (`paper3/replay/counts.sh`; Lake's own job total, `12014`, counts the cached Mathlib and dependency modules, none of which was compiled). The log has no line containing `error` and none containing `sorry`; its eleven `warning` lines are Lake noting that the lakefile's compatibility patches leave eleven dependencies with "local changes", and none of the eleven is Mathlib. Eight source files of the closure themselves carry nineteen `#print axioms` commands (seventeen files of the family's directory do, but nine of those are never built), so the build log records nineteen axiom lines as it goes, every one `[propext, Classical.choice, Quot.sound]`; among them `SqrtLogSpace/Final.lean`, lines 197–201, prints the axioms of `compiled_exact_count_and_sat`, `exact_count_and_sat`, `exact_count_eth`, `sat_polynomial_spa` and `original_sat_main`. The separate `#print axioms` run after the build gives, for the comparator's theorem,

```
'OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.exact_count_and_sat'
depends on axioms: [propext, Classical.choice, Quot.sound]
```

(`paper3/replay/logs/run2/axioms.txt`). The versions are `Lean (version 4.34.1, commit 5045d005..., Release)`, `Lake version 5.0.0-src+5045d00`, and Mathlib at `d13f23b723b8a846827`

from the clone's own manifest, with its object files from the Mathlib cache (8,908 files). The replay copy is at commit `4eaee875`, the same as the pristine clone, and the comparator file and the solution's files hash identically in both (`paper3/replay/logs/run2/hashes.txt`; the comparator statement's SHA-256 begins `4183b205`). The full build log is kept out of the repository by size; its SHA-256 (`d313f594...`) and its last 200 lines are committed.

So the sentence every later section relies on reads: *replayed*—the solution's closure builds from source at the pinned toolchain with upstream Mathlib, and the kernel reports the three standard axioms and nothing else for the theorem the comparator names. This moves the first three items of the inspection's trust boundary (Section 4): the build is no longer OpenAI's unpublished one, the `#print axioms` output is now on record, and the Mathlib revision was fetched from upstream and not patched.

5.3 The second kernel

The comparator's JSON for the family has `enable_nanoda: false`: OpenAI's check of this family ran the Lean kernel only. We ran a second kernel, in the same sandbox, after the build (`paper3/replay/nanoda.sh`, recorded in the README before it ran): `lean4export` (tag `v4.34.0`, commit `076e8e57`, built with the pinned `v4.34.1` toolchain so that it reads the object files the build produced) exported `exact_count_and_sat` and its transitive dependencies from the built environment—2,355,666,449 bytes of NDJSON, 43,633,742 lines, format version 3.1.0, Lean githash `5045d005...`, in 68 seconds—and `nanoda_lib` (nanoda contributors, 2026) (version 0.4.31, commit `8de7e0c4`, 2026-10-29; an external type checker for Lean 4 written in Rust, which shares no code with the Lean kernel) checked that export with `propext`, `Classical.choice` and `Quot.sound` as the permitted axioms and the `Nat` and `String` literal extensions on. Its output (`paper3/replay/logs/nanoda/nanoda_check.txt`):

```
Checked 95176 declarations with no errors
```

in 110 seconds on eight cores, exit code 0, after printing back the theorem's statement and the three axioms from its own parse of the export. The export contains exactly three `axiom` declarations (`grep -c '"axiom"'` on the file), so nothing else could have been permitted. The export file (SHA-256 `bdbc9afe...`, `paper3/replay/logs/nanoda/export_sha256.txt`) stays in the sandbox directory by size; the configuration, the build logs and the checker's output are committed.

5.4 What this does not claim

- The replay trusts the Lean kernel of the pinned toolchain (`v4.34.1`) and upstream Mathlib at the pinned revision; it moves the proof's trust from "OpenAI's unpublished build" to "a build we ran", not from the Lean kernel to anything else. The second kernel removes the Lean kernel from the list for the theorem's dependencies as exported, but adds `lean4export` (that it exports what the environment holds) and `nanoda_lib` (that it checks what it parses) to it; two independent checkers agreeing is evidence, not a proof that either is correct.
- It is not the comparator run. The comparator checks that the solution's exported statement matches the challenge's by `lean4export`; we did not run that tool chain, and the

statement-level match the paper relies on is the textual one of Section 4 (the 35 definitions and the theorem statement, identical character for character after whitespace normalisation, two of them up to the `noncomputable` keyword).

- The counts above are counts of lines in one build's log on one machine with the named versions; a different Lake would print different job totals (the 12,014 includes the cached Mathlib modules), and a different machine a different time.
- Nothing here bears on what the theorem *says* (Section 2) or on the manuscript's unverified exponents (Section 3); a replayed proof of a weak bound is a replayed proof of a weak bound.

6 What follows

Placeholder. Item 05 drafts this section: which hypotheses the verified statement refutes (ETH, SETH, Gap-ETH, #ETH, NSETH, the randomised forms) and how; in general terms, that a lower bound derived from ETH or SETH loses its premise while a statement "no $(c-\varepsilon)^k$ algorithm" is not itself refuted (the manuscript's §1.1.3); no example from any width parameter. Ends with what this does not claim".

7 For the practitioner

Placeholder. Item 06 drafts this section: a SAT solver and a model counter on random 3-CNF at $n = 20 \ldots 60$ against $2^{\rho(n)}$ and 2^n (figure labelled `fig:practitioner`, from `paper3/scripts/practitioner.py`); the verified guarantee is never the binding one on any machine. Ends with what this does not claim".

8 The statement in our Lean

Placeholder. Item 07 drafts this section: the restatement `SubexpCount` over the verbatim machine model under `lean/MOSPFormalization/ETH/`, the axiom audit, the byte-level rediff against the comparator file; no theorem of ours about the hypothesis is proved, and none is claimed. Ends with what this does not claim".

9 Related work and context

Placeholder. Item 08 drafts this section: the human audit of the August 2026 results (arXiv 2608.14673), the working group named in the collection's history, Impagliazzo–Paturi, Impagliazzo–Paturi–Zane, the comparator mechanism, nanoda; "we found no prior statement", never "first". Ends with what this does not claim".

10 Conclusion

Placeholder. Item 09 drafts this section: what is claimed and what is not; every entry of `paper3/ETH\_AMBIGUOUS.MD` that affects the text, listed as open by number. Ends with what this does not claim".

Data, code and proofs

Placeholder. Item 11 completes this statement: the repository, the scripts under `paper3/scripts/` that regenerate every number, the replay's commands and log hashes under `paper3/replay/`, the Lean names and the axiom audit under `paper3/lean/`, and the read-only clone of `https://github.com/openai/math` at commit `4eaee875`.

Acknowledgements

Placeholder. Item 11 writes the acknowledgements, with the disclosure of AI use in the form of `paper1/latex/sec8\_ack.tex`.

References

Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-SAT. *Journal of Computer and System Sciences*, 62(2):367–375, 2001. doi: 10.1006/jcss.2000.1727.

Lean FRO. comparator: checking a solution against a challenge statement. `https://github.com/leanprover/comparator`, 2026. Repository README; version and date recorded by item 08.

nanoda contributors. nanoda: a type checker for Lean 4 written in Rust. `https://github.com/ammkrn/nanoda_lib`, 2026. Repository README; author, version and date recorded by item 08.

OpenAI. The Exponential Time Hypothesis is False. OpenAI Math Release preprint OAI:The-Exponential-Time-Hypothesis-is-False-October-25-2026, 2026.

Appendix A Formal names

Every Lean name the paper cites, with its file under `lean/MOSPFormalization/ETH/`; the axiom audit is `paper3/lean/axiom_check.lean` with its committed log.

	result	Lean names
A.1	The comparator statement restated as a `Prop`, `SubexpCount` k, over the verbatim copy of the machine model; never proved, never an axiom	`MOSPFormalization.SubexpPathwidth.SubexpCount`
A.2	$\rho(n) = o(n)$: the verified exponent is subexponential in the limit	`MOSPFormalization.SubexpPathwidth.runtimeScale\_isLittleO`

Placeholder. Item 07 moves these declarations to `lean/MOSPFormalization/ETH/` and rewrites the names above to their new namespace.